DEEP GRAPH LIBRARY: A GRAPH-CENTRIC, HIGHLY-PERFORMANT PACKAGE FOR GRAPH NEURAL NETWORKS

Minjie Wang [2,3], Da Zheng [1], Zihao Ye [2], Quan Gan [2], Mufei Li [2], Xiang Song [2], Jinjing Zhou [2], Chao Ma [2], Lingfan Yu [3], Yu Gai [2], Tianjun Xiao [2], Tong He [2], George Karypis [1], Jinyang Li [3] & Zheng Zhang [2,4]
[1] Amazon Web Services, [2] AWS Shanghai AI Lab
[3] New York University, [4] NYU Shanghai

arXiv:1909.01315v2 [cs.LG] 25 Aug 2020

ABSTRACT

Advancing research in the emerging field of deep graph learning requires new tools to support tensor computation over graphs. In this paper, we present the design principles and implementation of Deep Graph Library (DGL)[1]. DGL distills the computational patterns of GNNs into a few generalized sparse tensor operations suitable for extensive parallelization. By advocating graph as the central programming abstraction, DGL can perform optimizations transparently. By cautiously adopting a framework-neutral design, DGL allows users to easily port and leverage the existing components across multiple deep learning frameworks. Our evaluation shows that DGL significantly outperforms other popular GNN-oriented frameworks in both speed and memory consumption over a variety of benchmarks and has little overhead for small scale workloads.

1 INTRODUCTION

Graph neural network (GNN) generalizes traditional deep learning to capture structural information in the data by modeling a set of node entities together with their relationships (edges). Its application range is broad, including molecules, social networks, knowledge graphs and recommender systems (Zitnik et al., 2018; Schlichtkrull et al., 2018; Hamilton et al., 2018; Ying et al., 2018), or in general any datasets that have structural information. As a vibrant and young field, accelerating research on GNN calls for developing domain packages that are simultaneously flexible and powerful for researchers, and efficient and performant for real-world applications.

Meeting both requirements are challenging: there are significant *semantic* gaps between the tensor-centric perspective of today's popular deep-learning (DL) frameworks and that of a graph, and *performance* gaps between the computation/memory-access patterns induced by the sparse nature of graphs and the underlying parallel hardware that are highly optimized for dense tensor operations.

This paper gives an overview of the design principles and implementation of Deep Graph Library (DGL), an open-source domain package specifically designed for researchers and application developers of GNN. Specifically, we make the following contributions:

- DGL distills the computational patterns of GNNs into a few user-configurable message-passing primitives; these primitives generalize sparse tensor operations and cover both the forward inference path and the backward gradient computing path. As such, they not only serve as the building blocks optimized for today's hardware, but also lay the foundation for future optimizations as well. In addition, DGL identifies and explores a wide range of parallelization strategies, leading to speed and memory efficiency.
- DGL makes graph the central programming abstraction. The graph abstraction allows DGL to simplify user programming by taking full control of the messy details of manipulating graph data.

[1]Project site: `https://www.dgl.ai`

- A full GNN application takes more than a GNN model; the other components (e.g. data preprocessing and feature extraction) are outside the scope of DGL. As such, DGL strives to be as framework neutral as possible. DGL runs on top of PyTorch (Paszke et al., 2019), TensorFlow (Abadi et al., 2016), MXNet (Chen et al., 2015) and leverages their capability as much it can, while minimizing the effort it takes to port a model across frameworks. Many choices we made are applicable to other domain packages that share the same aspiration.

The rest of the paper is organized as follows. We first introduce the backgrounds about GNN message passing in Sec. 2. We formulate these computations as two computational patterns – g-SpMM and g-SDMM, and discuss the parallelization strategies in Sec. 3. Sec. 4 describes the design and implementation of the DGL framework. We discuss some related works in Sec. 5 and evaluate DGL in Sec. 6.

2 GRAPH NEURAL NETWORKS AND MESSAGE PASSING

There have been a significant development in extending deep neural networks to non-euclidean data such as graphs and manifolds. Many efforts (Scarselli et al., 2009; Bruna et al., 2013; Defferrard et al., 2016; Kipf & Welling, 2017; Hamilton et al., 2017; Veličković et al., 2018) are made to formulate appropriate model architectures for learning on graphs, which gave birth to the Graph Neural Networks (GNNs) family. Recent studies (Gilmer et al., 2017; Battaglia et al., 2018) manage to unify different GNN variants into the **message passing paradigm**. Let $\mathcal{G}(\mathcal{V}, \mathcal{E})$ be a graph with nodes $\mathcal{V}$ and edges $\mathcal{E}$; Let $\mathbf{x}_v \in \mathbb{R}^{d_1}$ be the feature for node v, and $\mathbf{w}_e \in \mathbb{R}^{d_2}$ be the feature for edge (u, e, v)[2]. The message passing paradigm defines the following node-wise and edge-wise computation at step $t+1$:

$$\text{Edge-wise: } \mathbf{m}_e^{(t+1)} = \phi\left(\mathbf{x}_v^{(t)}, \mathbf{x}_u^{(t)}, \mathbf{w}_e^{(t)}\right), (u, e, v) \in \mathcal{E}. \tag{1}$$

$$\text{Node-wise: } \mathbf{x}_v^{(t+1)} = \psi\left(\mathbf{x}_v^{(t)}, \rho\left(\left\{\mathbf{m}_e^{(t+1)} : (u, e, v) \in \mathcal{E}\right\}\right)\right). \tag{2}$$

In the above equations, ϕ is a *message function* defined on each edge to generate a message by combining the edge feature with the features of its incident nodes; ψ is an *update function* defined on each node to update the node feature by aggregating its incoming messages using the *reduce function* ρ. In GNNs, the message and update functions are parameterized by neural network modules, and ρ can be any set function such as sum, mean, max/min, or even an LSTM network (Hamilton et al., 2017).

3 GNN MESSAGE PASSING AS GENERALIZED SPMM AND SDDMM.

There is a strong connection between the message passing paradigm to sparse matrix operations. For example, given the node feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$ and the adjacency matrix $\mathbf{A}$ of a graph, the node-wise computation in the graph convolutional network (GCN) (Kipf & Welling, 2017) is a sparse-dense matrix multiplication (SpMM) $\mathbf{Y} = \mathbf{A}\mathbf{X}$. For the edge-wise computation, many GNN models (Veličković et al., 2018; Kepner et al., 2016) calculate an attention weight on each edge. One popular formulation of calculating attention weight is by a dot product between the source and destination node features (Vaswani et al., 2017). This corresponds to a sampled dense-dense matrix multiplication (SDDMM) operation $\mathbf{W} = \mathbf{A} \odot (\mathbf{X}\mathbf{X}^T)$: semantically, it multiplies two dense matrices, followed by an element-wise multiplication with a sparse mask matrix, and output a sparse matrix.

An important characteristic of SDDMM is that it maps the representation of an edge's incident nodes to the representation on the edge. Similarly, SpMM aggregates the representation of a node's inbound edges into a node representation. Both of them can be extended. Given a graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$,

- A *generalized SDDMM* (g-SDDMM) defined on graph $\mathcal{G}$ with message function ϕ_m is a function

$$\text{g-SDDMM}_{\mathcal{G},\phi_m} : \mathbb{R}^{|\mathcal{V}| \times d_1}, \mathbb{R}^{|\mathcal{V}| \times d_2}, \mathbb{R}^{|\mathcal{E}| \times d_3} \mapsto \mathbb{R}^{|\mathcal{E}| \times d_4}$$

[2]We use a slightly different notation than a traditional (u, v) pair, where e is the ID associated with the edge.

where the output edge representations $\mathbf{M} = \text{g-SDDMM}_{\mathcal{G},\phi_m}(\mathbf{X}, \mathbf{Y}, \mathbf{W})$ are computed from the edges' own features, as well as features of their incident nodes:

$$\mathbf{m}_e = \phi_m(\mathbf{x}_u, \mathbf{y}_v, \mathbf{w}_e), \quad \forall (u, e, v) \in \mathcal{E}.$$

- A *generalized SpMM* (g-SpMM) defined on graph $\mathcal{G}$ with message function ϕ_z and reduce function ρ is a function

$$\text{g-SpMM}_{\mathcal{G},\phi_z,\rho} : \mathbb{R}^{|\mathcal{V}| \times d_1}, \mathbb{R}^{|\mathcal{V}| \times d_2}, \mathbb{R}^{|\mathcal{E}| \times d_3} \mapsto \mathbb{R}^{|\mathcal{V}| \times d_4}$$

  where the output node representations $\mathbf{Z} = \text{g-SpMM}_{\mathcal{G},\phi_z,\rho}(\mathbf{X}, \mathbf{Y}, \mathbf{W})$ are computed from the nodes' inbound edge features, the node features themselves, and the neighbor features:

$$\mathbf{z}_v = \rho(\{\phi_z(\mathbf{x}_u, \mathbf{y}_v, \mathbf{w}_e) : (u, e, v) \in \mathcal{E}\}), \quad \forall v \in \mathcal{V}.$$

These two primitives play an essential role in GNN computations; the forward path essentially applies a series of g-SpMM (and g-SDMM if attention is involved, as in GAT) to derive a stack of node representations. One can prove that the gradient of the objective function w.r.t. g-SDDMM and g-SpMM inputs can be expressed as another g-SDDMM and g-SpMM (see the supplementary materials for the full proof):

Theorem 1. *Given* $\mathbf{M} = \textit{g-SDDMM}_{\mathcal{G},\phi_m}(\mathbf{X}, \mathbf{Y}, \mathbf{W})$ *and* $\mathbf{Z} = \textit{g-SpMM}_{\mathcal{G},\phi_z,\rho}(\mathbf{X}, \mathbf{Y}, \mathbf{W})$ *and an objective function* $\mathcal{L} = \ell(\mathbf{M}, \mathbf{Z})$. *Then*

- *The partial derivative* $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ *can be computed by a g-SDDMM on graph* $\mathcal{G}$.
- *The partial derivative* $\frac{\partial \mathcal{L}}{\partial \mathbf{X}}$ *can be computed by a g-SpMM on the reverse graph* $\tilde{\mathcal{G}}(\mathcal{V}, \tilde{\mathcal{E}}), \tilde{\mathcal{E}} = \{(u, e, v) : (v, e, u) \in \mathcal{E}\}$.
- *The partial derivative* $\frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$ *can be computed by a g-SpMM on graph* $\mathcal{G}$.

The benefits of such formulation are two. First, consolidating all the GNN computations into two patterns lays a foundation for system optimizations such as parallelization and auto-tuning. Second, g-SpMM naturally avoids generating intermediate storage for messages, and g-SDDMM avoids copying node representations to edges. Later, we will show that such fused computation is the key reason for a superior training speed and memory efficiency (Sec. 6).

3.1 Parallelizing g-SpMM and g-SDDMM on tensorized hardware

Modern hardware like GPUs and TPUs utilizes large-scale multi-threading to achieve high throughput while hiding memory access latency. This requires the workload to have two characteristics. First, the computation-to-memory-access ratio must be high so that the cost of one memory operation is amortized over many floating point operations. Second, the workload should have sufficient parallelism to take advantage of the massive parallelization power in the hardware..

By these criteria, g-SpMM and g-SDDMM are inherently challenging workloads. First, each node's data is only used by its neighbors. With little data reuse, the resulting computation-to-memory-access ratio is low. Second, although there exist multiple ways to parallelize the g-SpMM and g-SDDMM operations – by node, edge or feature, different strategies have pros and cons and there is no jack of all trades. Feature parallelization lets each thread compute on one feature and different ones can run in parallel. Although it is free of synchronization, the parallelism is limited by hidden size. For parallelization on nodes and edges, the optimal performance depends on numerous factors (see Table 1), including the preferred storage format, the specific computation pattern (i.e., g-SpMM or g-SDDMM), fine-grained synchronization to ensure atomicity (if required), degree of parallelism, and thus is ultimately both data and model dependent. DGL's current strategy is based on heuristics – using node parallel for g-SpMM but edge parallel for g-SDDMM, and there is ample room to apply machine learning for performance optimization.

4 DGL system design

We now describe two important strategies we adopted in the development of DGL: 1) using graph as the central, user-friendly abstraction to allow deep optimization and 2) achieving maximum framework neutrality to enable seamless application integration.

	Node Parallel	**Edge Parallel**
Schedule	Each thread is in charge of the entire adjacency list of a node.	Each thread is in charge of one edge.
Viability	Any g-SpMM or g-SDDMM.	Any g-SDDMM and any g-SpMM with commutative and associative ρ.
Preferred Format	Compressed sparse row (CSR) due to fast lookup of adjacency list.	Coordinate list (COO) due to fast lookup of incident nodes.
Need for synchronization	No	No for g-SDDMM; g-SpMM requires atomic instructions for aggregating results to destination node memory.
Workload Distribution	Depend on node degrees	Balanced
Parallelism	Depend on number of nodes	Depend on number of edges

Table 1: Summary of the node and edge parallel strategies.

DGL

```
1  # init data
2  row, col = ...  # init graph
3  g = dgl.graph((row, col))
4  g.ndata['x'] = init_ndata()
5  g.edata['w'] = init_edata()
6  # create NN module
7  conv = dgl.nn.GraphConv(...)
8  loss = dgl.nn.DistMultLoss(...)
9  # extract subgraph
10 sg = dgl.sample_neighbors(
11   g, seeds, fanout=4)
12
13
14 # graph convolution
15 new_x = conv(sg, sg.ndata['x'], sg.edata['w'])
16 # negative sampling
17 subrow, _ = sg.adj(format='coo')
18 col_neg = randint(0, len(nid), size=len(col))
19 neg_g = dgl.graph((row, col_neg))
20 # compute loss
21 l = loss(neg_g, new_x)
```

PyTorch-Geometric

```
# init data
row, col = ... # init graph
adj = pyg.SparseTensor(row, col)
X = init_ndata()
W = init_edata()
# create NN module
conv = pyg.nn.GraphConv(...)
loss = pyg.nn.DistMultLoss(...)
# extract subgraph
subadj, nid, eid =
  pyg.sample_adj(adj, seeds, fanout=4)
subX = X[nid]
subW = W[eid]
# graph convolution
new_x = conv(subadj, subX, subW)
# negative sampling
sub_row = subadj.coo()[0]
col_neg = randint(0, len(nid), size=len(col))
neg_adj = pyg.SparseTensor(subrow, col_neg)
# compute loss
l = loss(neg_adj, new_x)
```

Figure 1: Computing graph convolution on a subgraph in DGL and PyG.

4.1 Graph as a first-class citizen

For a domain package designed for GNNs, it is natural to define graph as the central representation. While this is the consensus among different packages (Fey & Lenssen, 2019; Battaglia et al., 2018; Zhu et al., 2019; Alibaba, 2019), DGL differs in a number of places. First, it fully embraces an object-oriented programming style at the graph level. Second, recognizing the fact that research work on graphs are diverse and have developed a rich set of tools, it adopts an interface familiar to graph analytic experts. Third, the package exposes necessary low-level structures to advanced users when necessary. The fourth point, which we will discuss in Section 4.2, is its extensive use of tensor data structure to allow seamless integration with base DL frameworks.

Figure 1 compares the programming model of DGL and PyTorch Geometric (PyG) (Fey & Lenssen, 2019). In DGL, `DGLGraph` is the key data structure created by the `dgl.graph` API (line 3). Different models (e.g., `GraphConv` for graph convolution (Kipf & Welling, 2017), `GATConv` for graph attention model (Veličković et al., 2018)) operate on a `DGLGraph` directly (line 15); sampling, too, returns a `DGLGraph` object (line 10). The returned subgraph object automatically extracts the features needed, saving the effort to manually slice from tensors (line 12–13). Consolidating graph operations in an object-oriented manner not only improves software consistency, but also enables performance enhancement transparent to users. For example, DGL automatically switches to use CSR or CSC formats for g-SpMM depending on whether it is forward or backward propagation, and uses COO for g-SDDMM.

GNN Model	Change Type			Change /
	I	II	III	Total
GCN (Kipf & Welling, 2017)	4	12	14	30 / 103
GAT (Veličković et al., 2018)	4	26	7	37 / 95
GraphSage (Hamilton et al., 2017)	4	13	8	25 / 85
GIN (Xu et al., 2018)	4	4	0	8 / 37
SGC (Wu et al., 2019)	4	4	4	12 / 50

Table 2: The categorized number of lines of codes (LoC) need to change when porting a GNN layer in DGL from PyTorch to TensorFlow. Code comments are excluded.

Integrating graph with deep learning is a relatively young concept, but graph analytics has been a long-standing research field. There exist many sophisticated tools and packages. Many DGL APIs took inspiration from NetworkX (Hagberg et al., 2008) – the NumPy-equivalent python package in graph analytics. Examples are topological query APIs implanted as class member functions such as `g.in_degree` for getting node indegrees of a graph `g`. The two methods `g.edata` and `g.ndata` for accessing edge and node features are similarly inspired, with a dictionary-like interface that allows named tensors. Importantly, those APIs, while sharing naming convention with their NetworkX counterparts, have batched versions using tensor data structure. For example, NetworkX's `g.in_degree` only supports querying the degree of one node at a time, while DGL supports querying multiple nodes by providing a tensor of node IDs. Finally, we note that these APIs are more than a matter of convenience. For instance, a *heterogeneous graph* can have nodes or edges of different types, which can further have unaligned feature dimensions; it will be cumbersome to store them compactly in one tensor.

Finally, to maintain expressiveness and flexibility, it is important to expose internal structures so users can innovate beyond the APIs that `DGLGraph` offers. For instance, there have been a diverse number of studies (Kotnis & Nastase, 2017; Lukovnikov et al., 2017) on negative sampling strategies for the link prediction task. Users can craft these negative edges using the internal adjacency matrix of a DGLGraph object via the `g.adj` API (line 17–19 in Figure 1).

To define new GNN models, users can invoke the g-SpMM and g-SDDMM kernels via the `g.update_all(` ϕ, ρ`)` and `g.apply_edges(` ϕ`)` calls on a `DGLGraph`, with user-defined ϕ and ρ. In principle, a powerful compiler can parse any given functions and generate a fused kernel for execution. As the technique is yet to be developed (and thus is an active research), DGL provides a set of most common ϕ and ρ as built-ins and generates kernels for each of the combination. Note all these kernels avoid materializing edge data to save memory, which is important for all GNN models where edge features are needed (Sec. 6). For more complex user-defined functions (e.g., LSTM as a reduction function), DGL gathers node features to edges so users can compute messages in a batch. For the reduce phase, DGL groups nodes of the same degree into one bucket so that the received messages in each bucket can be stored in a dense tensor. DGL then invokes the user-defined reduce function repetitively on each bucket. This *catch-all* strategy makes it easy for quickly prototyping model ideas on graphs of small sizes.

4.2 Framework-neutral design

A natural way to build a domain package is to build it on top of *one* of the DL frameworks (e.g., PyTorch, TensorFlow, and MXNet). These mature frameworks already provide high-performance differentiable dense tensor operators, rich neural network modules and optimizers; there is scarcely any reason to reinvent the wheel. DGL makes a conscious decision to extend *multiple* frameworks and, consequently, to be as framework-neutral as possible. Our belief is that a real-world, end-to-end GNN application will require other modules outside of GNN, and that they can be, or have already been, implemented in any framework of users' choice. In addition, users may favor a particular framework simply because of its unique features.

Note that being framework-neutral is different from framework-agnostic. That is, while DGL has both PyTorch and TensorFlow backends, a PyTorch DGL model still needs to be modified if it is to be run in TensorFlow. Being completely framework-agnostic requires putting a shim over all conceivable operators across different frameworks, the cost of which is prohibitive. Instead, we adopt

a practical approach and reduce framework dependencies as much as possible, while providing clear guidelines as where the changes are to be made. Importantly, DGL can achieve a high degree of framework neutrality in part due to the abstraction and implementation of `DGLGraph` . As we shall describe below and quantify through the models that we have implemented, the changes are often local and trivial.

A complete GNN application includes data loading and preprocessing, GNN model setup, training loop and evaluation. In theory, they are all framework dependent. The goal of our design is to make model specification as portable as possible. Versions of the same model for different frameworks differ in three categories: **(I)** model class inheritance (e.g., using `tensorflow.keras.layer.Layer` instead of `torch.nn.Module`); **(II)** sub-modules used inside the model and parameter initialization; **(III)** framework-specific operators (e.g., using `tensorflow.matmul` instead of `torch.matmul`). A mini-porting guide and example codes are included in the supplementary materials.

Table 2 shows the number of lines of code to change when we port several GNN layers from PyTorch to TensorFlow in DGL. They account for roughly 20%–40% of the entire model implementations. Most of them are trivial modifications and are easy for developers versed in both frameworks. Importantly, all graph-related operations are unified and stay identical in different versions.

To achieve this level of framework neutrality with a minimum performance impact, DGL must decide what functionalities to delegate and re-direct to base frameworks, and otherwise judiciously take over the control. We summarize the main principles below; most of them shall be applicable to other domain packages that share the same aspiration.

Owning the minimum & the critical. From our experience, a domain package must maintain control at places where performance or usability matters the most. For DGL, it means sparse tensor storage management and operations. This leads to the decision of defining `DGLGraph` (see Section 4.1). The first release of DGL used dense operations from frameworks to express sparse operations in GNNs and performed poorly. We decided to implemented sparse operations ourselves.

Leverage and delegate otherwise. Most of DGL's APIs take framework tensors as input and perform dense operations on them. DGL defines a shim to map dense tensor operations to their framework-specific implementations. For instance, summing up the hidden state of all nodes is a common readout function for graph-level classification. To batch this readout operation we define a shim function `unsorted_1d_segment_sum` , which translates to `unsorted_segment_sum` in Tensorflow and `scatter_add` in PyTorch. Such remapping is in spirit similar to ONNX (onn, 2018), but is designed specifically for DGL.

To enable auto-differentiation, all computation involving node/edge features must be expressed with differentiable functions and operators. DGL defines custom functions that directly take the `DGLGraph` as well as node/edge features as inputs, and return node/edge features as outputs. These operators are then registered as PyTorch/Tensorflow/MXNet auto-differentiable functions.

DGL also takes advantage of DLPack (dlp, 2017), an open-source in-memory tensor structure specification for sharing tensors among deep learning frameworks, to directly process and return DL framework tensors without data copying. Many frameworks, including Pytorch, MXNet, and TensorFlow, natively support DLPack.

The above functionality calls for memory allocation and management. DGL delegates memory management to the base frameworks. A base framework usually implements sophisticated memory management (e.g., to reduce memory allocation overhead and memory consumption), which is especially important for GPU memory. Because the output shape of a graph kernel is well determined before execution, DGL calculates the output memory size, allocates memory from the frameworks for the output tensors and pass the memory to the kernel for execution.

5 Related Work

Due to the rising interests in GNNs, frameworks designed specifically for expressing and accelerating GNNs are developed. PyTorch-Geometric (PyG) (Fey & Lenssen, 2019) is an extension for geometric deep learning to the PyTorch framework. PyG's programming model is centered around sparse tensor abstraction. During message passing, it first *gathers* node features to edges, applies user-defined

message function and then *scatters* them to the target nodes for aggregation. This scatter-gather pattern is inefficient due to generating large intermediate message tensors. GraphNet (Battaglia et al., 2018) and AliGraph (Zhu et al., 2019) are two TensorFlow packages for building GNN models. Both frameworks allow customizable message functions but the reducers are limited to TensorFlow's operators for segment reduction. Euler (Alibaba, 2019) focuses on sampling-based mini-batching training on large graphs but lacks GPU support. NeuGraph (Ma et al., 2019) accelerates GNN training by partitioning graphs to multiple GPUs. All of these systems are tied to one specific base DL framework.

There is a long line of work for optimizing sparse matrix operators such as sparse matrix-vector multiplication (SpMV) or sparse matrix-matrix multiplication (SpMM) on both CPU and GPU. These techniques range from studying and innovating new sparse matrix formats (Bell & Garland, 2008; Filippone et al., 2017), advanced parallel pattern (Yang et al., 2018) to tiling and reordering (Yang et al., 2011; Baskaran & Bordawekar, 2009) in the context of graph analytics (Ashari et al., 2014; Wang et al., 2016) or scientific computing applications (LeVeque, 2007), just to name a few. Our work formally connects the area to GNN applications through the notions of generalized SpMM and SDDMM. We present the emerging challenges and hope to open up new innovations in this domain.

6 EVALUATION

In this section, we compare DGL with other popular GNN frameworks: PyTorch-Geometric v1.5.0 (PyG) with PyTorch v1.5.0 as backend and GraphNet v1.1.0 with TensorFlow v2.2.0 as backend.[3]

6.1 TRAINING SPEED

We consider two benchmark tasks: node classification and link prediction, and two training methods: full graph training and mini-batch training. Node classification datasets include the REDDIT graph from (Hamilton et al., 2017), the OGBN-ARXIV, OGBN-PROTEIN, and OGBN-PRODUCT graphs from the Open Graph Benchmarks (OGB) (Hu et al., 2020). For link prediction, we use benchmarks from MOVIELENS(ML) (Harper & Konstan, 2015), the OGBL-CITATION and OGBL-PPA graphs from OGB.

To demonstrate the generality of DGL's optimizations, we benchmark a variety of state-of-the-art GNN models, including GCN (Kipf & Welling, 2017), GraphSAGE (Hamilton et al., 2017), GAT (Veličković et al., 2018), R-GCN (Schlichtkrull et al., 2017) and GCMC (Berg et al., 2017). All the node classification tasks use cross entropy loss on the node representations learned by the GNN models while the link prediction tasks perform edge predictions by computing the dot-product of the source and destination node representations. For mini-batch training, we experiment with two sampling methods: neighbor sampling (NS) (Hamilton et al., 2017) and cluster sampling (CS) (Chiang et al., 2019). The supplementary material includes additional details about the datasets and model configurations.

All experiments record the training time of one epoch averaged over 10 runs. For full graph training, we measure the training time on both CPU and GPU. The testbeds are one AWS EC2 p3.2xlarge instance (one NVidia V100 GPU with 16GB GPU RAM and 8 VCPUs) and one m5n.16xlarge instance (64 VCPUs and 256GB RAM) for experiments on GPU and CPU respectively. For mini-batch training, we perform sampling on CPU and copy the sampled subgraphs and features to GPU for training. The testbed is a p3.2xlarge instance.

Table 3 shows the results of full graph training. For GraphSAGE on GPU, both DGL and PyG use the vendor-provided cuSPARSE (Naumov et al., 2010) library for computing SpMM so the performance is similar. DGL is slower by a small margin (2–11%) due to framework overhead. For GAT, DGL is 1.68× faster than PyG on OGBN-ARXIV because DGL's g-SpMM kernel avoids generating message tensors while PyG's scatter-gather kernel does. This also explains PyG running out-of-memory on OGBN-PROTEIN due to the graph being the densest one among all and having edge features. Link prediction benchmarks show similar results. DGL is slower on small graphs (e.g., ML-100K) but is 1.83× faster on ML-1M. DGL can train on ML-10M while PyG runs out of memory. On CPU, DGL outperforms PyG on all benchmarks by 1.9×–64×. This is attributing to the high CPU utilization

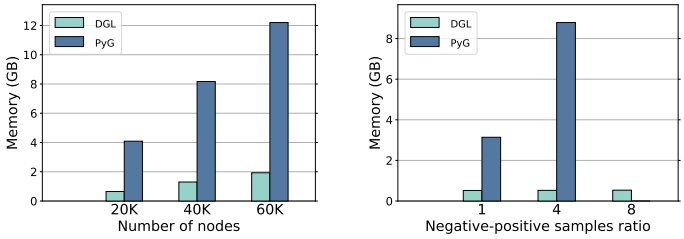
[3]Benchmark scripts are available at `https://github.com/dglai/dgl-0.5-benchmark/`

Dataset	Model	CPU		GPU	
		DGL	PyG	DGL	PyG
Node Classification					
REDDIT	SAGE	**13.80**	99.47	0.432	**0.403**
REDDIT	GAT	**9.15**	OOM	0.718	OOM
OGBN-ARXIV	SAGE	**3.31**	8.389	0.104	**0.098**
OGBN-ARXIV	GAT	**1.237**	43.21	**0.086**	0.234
OGBN-PROTEIN	R-GCN	**26.31**	373.8	**0.706**	OOM
Link Prediction					
ML-100K	GCMC	**0.064**	1.569	0.021	**0.012**
ML-1M	GCMC	**0.351**	40.47	**0.045**	0.103
ML-10M	GCMC	**5.08**	OOM	**0.412**	OOM

Table 3: Epoch running time in seconds (full graph training). OOM means out-of-memory.

Dataset	Model	DGL	PyG
Node Classification			
REDDIT	SAGE w/ NS	**19.90**	20.45
REDDIT	GAT w/ NS	**21.07**	21.89
OGBN-PRODUCT	SAGE w/ NS	**33.34**	35.00
OGBN-PRODUCT	GAT w/ NS	**67.0**	187.0
OGBN-PRODUCT	SAGE w/ CS	8.887	**8.614**
OGBN-PRODUCT	GAT w/ CS	**14.50**	58.36
Link Prediction			
OGBL-CITATION	GCN w/ CS	**5.772**	6.287
OGBL-CITATION	GAT w/ CS	**6.081**	8.290
OGBL-PPA	GCN w/ CS	**5.782**	6.421
OGBL-PPA	GAT w/ CS	**6.224**	8.198

Table 4: Epoch running time in seconds for mini-batch training using neighbor sampling (NS) and cluster sampling (CS).



(a) (b)

Figure 2: Memory usage of PyG and DGL. (a) GAT on synthetic graphs; (b) GCN w/ CS on OGBL-CITATION.

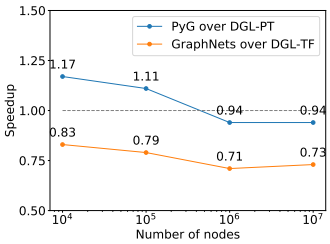


Figure 3: Raw framework speedup of PyG and GraphNets over DGL.

(50%) of DGL's g-SpMM and g-SDDMM kernels using multi-threading compared with PyG's (only 10%). The large gap of ML-1M is further caused by the huge intermediate message tensor, resulting in a lot of time spent in memory traffic.

Table 4 evaluates the performance of mini-batch training. Compared with full graph training, the total training time also depends on the cost in sample preparation including the sampling operations and data movement from CPU to GPU. For neighbor sampling (NS), sample generation and data movement can occupy up to 85% of the total training time, which explains why DGL and PyG have similar performance. By contrast, cluster sampling (CS) is much faster and the benefit from DGL's optimized kernels gives an 1.56× speedup for training GAT. DGL beats PyG in all link prediction benchmarks due to the use of g-SDDMM kernel in computing predictions on edges.

6.2 MEMORY CONSUMPTION

To illustrate the advantage of DGL's g-SpMM and g-SDDMM kernels in reducing memory traffic, we further studied the memory usage of DGL and PyG. We trained a 3-layer GAT model with one attention head on a set of synthetic graphs of different scales. The average degree is fixed at 20 so the number of edges grows linearly with the number of nodes. Figure 2a shows that PyG consumes 6.3× more memory than DGL and runs out of memory on graphs of more than 60K nodes. DGL manages to keep a low memory footprint due to its g-SpMM kernel fusing the message computation with aggregation. We further investigate the case of link prediction using a 3-layer GCN model with cluster sampling on the OGB-CITATION dataset. The model computes a prediction on each edge by performing a dot-product of its source and destination node representations, which is a typical SDDMM operation. Figure 2b shows the memory usage by increasing the number of negative edges per positive ones at each mini-batch. DGL's memory consumption stays the same regardless of the number of negative samples while PyG quickly runs out of memory. Since negative sampling is universal in link prediction tasks, we expect the phenomenon to appear in other benchmarks as well.

6.3 FRAMEWORK OVERHEAD

We compared DGL's framework overhead with both PyG and GraphNets using PyTorch and TensorFlow as backends, respectively. In order to eliminate the impact of message passing kernels, we trained an one-layer GCN over a synthetically generated chain graph and measured the epoch time. We varied the number of nodes and plotted the speedup of PyG and GraphNets over DGL in Figure 3. Ideally, the speedup should be one. We observed a 17% overhead compared with PyG when the graph is very small, but as the graph size increases, the overhead becomes negligible. The overhead is due to DGL registering message passing kernels via Python to keep implementation independent of frameworks while PyG can register them in C++. Interestingly, DGL-TF is faster than GraphNets, which uses native TensorFlow operators, demonstrating the viability of a framework-neutral package with low overhead.

7 CONCLUSION

We present Deep Graph Library (DGL), a system specialized for deep learning models on graphs. DGL identifies the connection between sparse matrix computation and the message passing paradigm in graph neural networks, and consolidates these operations into generalized sparse-dense matrix multiplication (g-SpMM) and sampled dense-dense matrix multiplication (g-SDDMM). DGL explores a wide range of parallelization strategies, leading to its superior speed and memory efficiency. DGL presents two design principles. By having graph as the core programming abstraction, DGL can hide cumbersome details from users and perform optimization transparently. DGL's lessons in designing a framework-neutral domain package with low overhead shall also be applicable to other packages of the same kind.

REFERENCES

Dlpack. `https://github.com/dmlc/dlpack`, 2017.

Open neural network exchange format. `https://github.com/onnx/onnx`, 2018.

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *OSDI*, volume 16, pp. 265–283, 2016.

Alibaba. Euler. `https://github.com/alibaba/euler`, 2019.

Arash Ashari, Naser Sedaghati, John Eisenlohr, Srinivasan Parthasarath, and P Sadayappan. Fast sparse matrix-vector multiplication on gpus for graph applications. In *SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 781–792. IEEE, 2014.

Muthu Manikandan Baskaran and Rajesh Bordawekar. Optimizing sparse matrix-vector multiplication on gpus. *IBM Research Report RC24704*, (W0812-047), 2009.

Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.

Nathan Bell and Michael Garland. Efficient sparse matrix-vector multiplication on cuda. Technical report, Nvidia Technical Report NVR-2008-004, Nvidia Corporation, 2008.

Rianne van den Berg, Thomas N Kipf, and Max Welling. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263*, 2017.

Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. Spectral networks and locally connected networks on graphs. *arXiv preprint arXiv:1312.6203*, 2013.

Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv preprint arXiv:1512.01274*, 2015.

Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 257–266, 2019.

Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in neural information processing systems*, pp. 3844–3852, 2016.

Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. *CoRR*, abs/1903.02428, 2019.

Salvatore Filippone, Valeria Cardellini, Davide Barbieri, and Alessandro Fanfarillo. Sparse matrix-vector multiplication on gpgpus. *ACM Transactions on Mathematical Software (TOMS)*, 43(4): 1–49, 2017.

Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, 2017.

Aric Hagberg, Pieter Swart, and Daniel S Chult. Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2008.

Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, pp. 1024–1034, 2017.

Will Hamilton, Payal Bajaj, Marinka Zitnik, Dan Jurafsky, and Jure Leskovec. Embedding logical queries on knowledge graphs. In *Advances in Neural Information Processing Systems*, pp. 2030–2041, 2018.

F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.

Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *arXiv preprint arXiv:2005.00687*, 2020.

Jeremy Kepner, Peter Aaltonen, David Bader, Aydin Buluç, Franz Franchetti, John Gilbert, Dylan Hutchison, Manoj Kumar, Andrew Lumsdaine, Henning Meyerhenke, et al. Mathematical foundations of the graphblas. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)*, pp. 1–9. IEEE, 2016.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Bhushan Kotnis and Vivi Nastase. Analysis of the impact of negative sampling on link prediction in knowledge graphs. *arXiv preprint arXiv:1708.06816*, 2017.

Randall J LeVeque. *Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems*, volume 98. Siam, 2007.

Denis Lukovnikov, Asja Fischer, Jens Lehmann, and Sören Auer. Neural network-based question answering over knowledge graphs on word and character level. In *Proceedings of the 26th international conference on World Wide Web*, pp. 1211–1220, 2017.

Lingxiao Ma, Zhi Yang, Youshan Miao, Jilong Xue, Ming Wu, Lidong Zhou, and Yafei Dai. Neugraph: parallel deep neural network computation on large graphs. In *2019 {USENIX} Annual Technical Conference ({USENIX}{ATC} 19)*, pp. 443–458, 2019.

M Naumov, LS Chien, P Vandermersch, and U Kapasi. Cusparse library. In *GPU Technology Conference*, 2010.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, pp. 8024–8035, 2019.

Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in neural information processing systems*, pp. 5099–5108, 2017.

Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. *arXiv preprint arXiv:1703.06103*, 2017.

Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference*, pp. 593–607. Springer, 2018.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pp. 5998–6008, 2017.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks. *International Conference on Learning Representations*, 2018.

Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. Gunrock: A high-performance graph processing library on the gpu. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 1–12, 2016.

Felix Wu, Tianyi Zhang, Amauri Holanda de Souza Jr, Christopher Fifty, Tao Yu, and Kilian Q Weinberger. Simplifying graph convolutional networks. *arXiv preprint arXiv:1902.07153*, 2019.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.

Carl Yang, Aydın Buluç, and John D Owens. Design principles for sparse matrix multiplication on the gpu. In *European Conference on Parallel Processing*, pp. 672–687. Springer, 2018.

Xintian Yang, Srinivasan Parthasarathy, and Ponnuswamy Sadayappan. Fast sparse matrix-vector multiplication on gpus: implications for graph mining. *arXiv preprint arXiv:1103.2405*, 2011.

Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. *KDD*, 2018.

Rong Zhu, Kun Zhao, Hongxia Yang, Wei Lin, Chang Zhou, Baole Ai, Yong Li, and Jingren Zhou. Aligraph: a comprehensive graph neural network platform. *Proceedings of the VLDB Endowment*, 12(12):2094–2105, 2019.

Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018.

APPENDIX A NOMENCLATURES

In the Appendix we adopt the following nomenclatures for representing different mathematical objects:

- a: a scalar
- $\mathbf{a}$: a (column) vector
- $\mathbf{A}$: a matrix
- $\mathbf{a}_i$: the i-th row of matrix $\mathbf{A}$
- $\mathbb{R}^{m\times n}$: the set of real matrices with m rows and n columns.
- $\{x : y\}$ the set with all mathematical objects x that satisfies condition y.
- $f(\cdot,\cdot,\dots)$: a function.
- $f : \mathcal{X} \mapsto \mathcal{Y}$: a function that maps from set $\mathcal{X}$ to $\mathcal{Y}$
- $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ or $\nabla_{\mathbf{x}}\mathbf{y}$: the Jacobian of $\mathbf{y}$ with respect to $\mathbf{x}$, in numerator layout.
- $\frac{\partial f}{\partial \mathbf{x}}$: the partial derivative of scalar function f with respect to vector $\mathbf{x}$, in numerator layout. Note that in numerator layout $\frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} & \cdots \end{bmatrix}$ is a row vector.
- $\frac{\partial f}{\partial \mathbf{A}}$: the partial derivative of scalar function f with respect to matrix $\mathbf{A}$.
- $[\mathbf{A};\mathbf{B};\cdots] = [\mathbf{A} \quad \mathbf{B} \quad \dots]$: horizontal concatenation of matrices.

APPENDIX B GRADIENT OF G-SPMM AND G-SDDMM

We go by reviewing the definition of g-SpMM and g-SDDMM:

Definition 1. *A generalized SDDMM (g-SDDMM) defined on graph $\mathcal{G}$ with message function ϕ_m is a function*

$$\textit{g-SDDMM}_{\mathcal{G},\phi_m} : \mathbb{R}^{|\mathcal{V}|\times d_1}, \mathbb{R}^{|\mathcal{V}|\times d_2}, \mathbb{R}^{|\mathcal{E}|\times d_3} \mapsto \mathbb{R}^{|\mathcal{E}|\times d_4}$$

where the output edge representations $\mathbf{M} = \textit{g-SDDMM}_{\mathcal{G},\phi_m}(\mathbf{X},\mathbf{Y},\mathbf{W})$ are computed from the edges' own features, as well as features of their incident nodes:

$$\mathbf{m}_e = \phi_m(\mathbf{x}_u, \mathbf{y}_v, \mathbf{w}_e), \quad \forall (u,e,v) \in \mathcal{E}.$$

Definition 2. *A generalized SpMM (g-SpMM) defined on graph $\mathcal{G}$ with message function ϕ_z and reduce function ρ is a function*

$$\textit{g-SpMM}_{\mathcal{G},\phi_z,\rho} : \mathbb{R}^{|\mathcal{V}|\times d_1}, \mathbb{R}^{|\mathcal{V}|\times d_2}, \mathbb{R}^{|\mathcal{E}|\times d_3} \mapsto \mathbb{R}^{|\mathcal{V}|\times d_4}$$

where the output node representations $\mathbf{Z} = \textit{g-SpMM}_{\mathcal{G},\phi_z,\rho}(\mathbf{X},\mathbf{Y},\mathbf{W})$ are computed from the nodes' inbound edge features, the node features themselves, and the neighbor features:

$$\mathbf{z}_v = \rho(\{\phi_z(\mathbf{x}_u, \mathbf{y}_v, \mathbf{w}_e) : (u,e,v) \in \mathcal{E}\}), \quad \forall v \in \mathcal{V}.$$

We also review the formal definition of the reverse graph.

Definition 3. *Given the graph $\mathcal{G} = (\mathcal{V},\mathcal{E})$, the reverse graph $\tilde{\mathcal{G}} = (\mathcal{V},\tilde{\mathcal{E}})$, where $\tilde{\mathcal{E}} = \{(u,e,v) : (v,e,u) \in \mathcal{E}\}$ contains the corresponding edges reversing directions.*

We then show that the gradient of g-SpMM and g-SDDMM functions can also be expressed as g-SpMM and g-SDDMM functions.

Lemma 1. *Assume we are given the g-SDDMM function*

$$\mathbf{M} = \textit{g-SDDMM}_{\mathcal{G},\phi_m}(\mathbf{X},\mathbf{Y},\mathbf{W})$$

defined on graph $\mathcal{G}$ with message function ϕ_m and the objective function $\mathcal{L} = \ell(\mathbf{M})$. There exists a function

$$\phi'_w : \mathbb{R}^{|\mathcal{V}|\times d_1}, \mathbb{R}^{|\mathcal{V}|\times d_2}, \mathbb{R}^{|\mathcal{E}|\times (d_3+d_4)} \mapsto \mathbb{R}^{|\mathcal{E}|\times d_3}$$

such that

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \textit{g-SDDMM}_{\mathcal{G},\phi'_w}\left(\mathbf{X},\mathbf{Y},\left[\mathbf{W};\frac{\partial \mathcal{L}}{\partial \mathbf{M}}\right]\right)$$

Proof. By chain rule, for each $(u, e, v) \in \mathcal{E}$ we have:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_e} = \frac{\partial \mathcal{L}}{\partial \mathbf{m}_e} \frac{\partial \mathbf{m}_e}{\partial \mathbf{w}_e} = \frac{\partial \mathcal{L}}{\partial \mathbf{m}_e} \nabla_{\mathbf{w}_e} \phi_m \left(\mathbf{x}_u, \mathbf{y}_v, \mathbf{w}_e \right)$$

□

Lemma 2. *Assume we are given the g-SDDMM function*

$$\mathbf{M} = \textit{g-SDDMM}_{\mathcal{G}, \phi_m} \left(\mathbf{X}, \mathbf{Y}, \mathbf{W} \right)$$

defined on graph $\mathcal{G}$ with message function ϕ_m and the objective function $\mathcal{L} = \ell(\mathbf{M})$. There exists functions ϕ'_x and ϕ'_y

$$\phi'_x : \mathbb{R}^{|\mathcal{V}| \times d_1}, \mathbb{R}^{|\mathcal{V}| \times d_2}, \mathbb{R}^{|\mathcal{E}| \times (d_3 + d_4)} \mapsto \mathbb{R}^{|\mathcal{V}| \times d_1}$$
$$\phi'_y : \mathbb{R}^{|\mathcal{V}| \times d_1}, \mathbb{R}^{|\mathcal{V}| \times d_2}, \mathbb{R}^{|\mathcal{E}| \times (d_3 + d_4)} \mapsto \mathbb{R}^{|\mathcal{V}| \times d_2}$$

such that

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \textit{g-SpMM}_{\tilde{\mathcal{G}}, \phi'_x, \sum} \left(\mathbf{X}, \mathbf{Y}, \left[\mathbf{W}; \frac{\partial \mathcal{L}}{\partial \mathbf{M}} \right] \right)$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{Y}} = \textit{g-SpMM}_{\mathcal{G}, \phi'_y, \sum} \left(\mathbf{X}, \mathbf{Y}, \left[\mathbf{W}; \frac{\partial \mathcal{L}}{\partial \mathbf{M}} \right] \right)$$

where $\tilde{\mathcal{G}}$ represents the reverse graph of $\mathcal{G}$, and $\sum$ denotes the summation as a reduce function of the g-SpMMs.

Proof. By chain rule, for each $u, v \in \mathcal{V}$ we have:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \mathbf{x}_u} &= \sum_{e', v' : (u, e', v') \in \mathcal{E}} \frac{\partial \mathcal{L}}{\partial \mathbf{m}_{e'}} \frac{\partial \mathbf{m}_{e'}}{\partial \mathbf{x}_u} \\
&= \sum_{e', v' : (u, e', v') \in \mathcal{E}} \frac{\partial \mathcal{L}}{\partial \mathbf{m}_{e'}} \nabla_{\mathbf{x}_u} \phi_m \left(\mathbf{x}_u, \mathbf{y}_{v'}, \mathbf{w}_{e'} \right) \\
&= \underbrace{\sum_{e', v' : (v', e', u) \in \tilde{\mathcal{E}}}}_{\text{reduce function}} \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{m}_{e'}} \nabla_{\mathbf{x}_u} \phi_m \left(\mathbf{x}_u, \mathbf{y}_{v'}, \mathbf{w}_{e'} \right)}_{\text{message function on the reverse graph}} \\
\frac{\partial \mathcal{L}}{\partial \mathbf{y}_v} &= \sum_{u', e' : (u', e', v) \in \mathcal{E}} \frac{\partial \mathcal{L}}{\partial \mathbf{m}_{e'}} \frac{\partial \mathbf{m}_{e'}}{\partial \mathbf{y}_v} \\
&= \underbrace{\sum_{u', e' : (u', e', v) \in \mathcal{E}}}_{\text{reduce function}} \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{m}_{e'}} \nabla_{\mathbf{y}_v} \phi_m \left(\mathbf{x}_{u'}, \mathbf{y}_v, \mathbf{w}_{e'} \right)}_{\text{message function}}
\end{aligned}$$

□

Lemma 3. *Assume we are given the g-SpMM function*

$$\mathbf{Z} = \textit{g-SpMM}_{\mathcal{G}, \phi_z, \rho} \left(\mathbf{X}, \mathbf{Y}, \mathbf{W} \right)$$

defined on graph $\mathcal{G}$ with message function ϕ_z and reduce function ρ and the objective function $\mathcal{L} = \ell(\mathbf{Z})$. There exists a function ϕ'_w

$$\phi'_w : \mathbb{R}^{|\mathcal{V}| \times d_1}, \mathbb{R}^{|\mathcal{V}| \times (d_2 + d_4)}, \mathbb{R}^{|\mathcal{E}| \times d_3} \mapsto \mathbb{R}^{|\mathcal{E}| \times d_3}$$

such that

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \textit{g-SDDMM}_{\mathcal{G}, \phi'_w} \left(\mathbf{X}, \left[\mathbf{Y}; \frac{\partial \mathcal{L}}{\partial \mathbf{Z}} \right], \mathbf{W} \right)$$

Proof. Let $\mathbf{m}_e = \phi_z(\mathbf{x}_u, \mathbf{y}_v, \mathbf{w}_e)$ for all $(u, e, v) \in \mathcal{E}$. Since $\mathbf{m}_e$ only participates in computation of $\mathbf{z}_v$, we have

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_e} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_v} \frac{\partial \mathbf{z}_v}{\partial \mathbf{m}_e} \frac{\partial \mathbf{m}_e}{\partial \mathbf{w}_e} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_v} \nabla_{\mathbf{m}_e} \rho\left(\{\mathbf{m}_{e'} : (u', e', v) \in \mathcal{E}\}\right) \nabla_{\mathbf{w}_e} \phi\left(\mathbf{x}_u, \mathbf{y}_v, \mathbf{w}_e\right)$$

□

Lemma 4. *Assume we are given the g-SpMM function*

$$\mathbf{Z} = \textit{g-SpMM}_{\mathcal{G}, \phi_z, \rho}\left(\mathbf{X}, \mathbf{Y}, \mathbf{W}\right)$$

defined on graph $\mathcal{G}$ with message function ϕ_z and reduce function ρ and the objective function $\mathcal{L} = \ell(\mathbf{Z})$. There exists functions ϕ'_x and ϕ'_y

$$\phi'_x : \mathbb{R}^{|\mathcal{V}| \times d_1}, \mathbb{R}^{|\mathcal{V}| \times (d_2 + d_4)}, \mathbb{R}^{|\mathcal{E}| \times d_3} \mapsto \mathbb{R}^{|\mathcal{V}| \times d_1}$$
$$\phi'_y : \mathbb{R}^{|\mathcal{V}| \times d_1}, \mathbb{R}^{|\mathcal{V}| \times (d_2 + d_4)}, \mathbb{R}^{|\mathcal{E}| \times d_3} \mapsto \mathbb{R}^{|\mathcal{V}| \times d_2}$$

and set functions ρ'_x, ρ'_y, such that

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \textit{g-SpMM}_{\tilde{\mathcal{G}}, \phi'_x, \rho'_x}\left(\mathbf{X}, \left[\mathbf{Y}; \frac{\partial \mathcal{L}}{\partial \mathbf{Z}}\right], \mathbf{W}\right)$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{Y}} = \textit{g-SpMM}_{\mathcal{G}, \phi'_y, \rho'_y}\left(\mathbf{X}, \left[\mathbf{Y}; \frac{\partial \mathcal{L}}{\partial \mathbf{Z}}\right], \mathbf{W}\right)$$

where $\tilde{\mathcal{G}}$ represents the reverse graph of $\mathcal{G}$.

Proof. We first show the correctness for $\frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$.

Let $\mathbf{m}_e = \phi_z(\mathbf{x}_u, \mathbf{y}_v, \mathbf{w}_e)$ for all $(u, e, v) \in \mathcal{G}$. Since $\mathbf{y}_v$ only takes part in the computation of $\mathbf{z}_v$, we have

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \mathbf{y}_v} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}_v} \frac{\partial \mathbf{z}_v}{\partial \mathbf{y}_v} \\
&= \frac{\partial \mathcal{L}}{\partial \mathbf{z}_v} \sum_{u', e' : (u', e', v) \in \mathcal{E}} \frac{\partial \mathbf{z}_v}{\partial \mathbf{m}_{e'}} \frac{\partial \mathbf{m}_{e'}}{\partial \mathbf{y}_v} \\
&= \sum_{u', e' : (u', e', v) \in \mathcal{E}} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_v} \nabla_{\mathbf{m}_{e'}} \rho\left(\{\mathbf{m}_{e''} : (u'', e'', v) \in \mathcal{E}\}\right) \nabla_{\mathbf{y}_v} \phi_z\left(\mathbf{x}_{u'}, \mathbf{y}_v, \mathbf{w}_{e'}\right)
\end{aligned}$$

To derive $\frac{\partial \mathcal{L}}{\partial \mathbf{X}}$, we need to sum over all successors of u:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \mathbf{x}_u} &= \sum_{e', v' : (u, e', v') \in \mathcal{E}} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{v'}} \frac{\partial \mathbf{z}_{v'}}{\partial \mathbf{x}_u} \\
&= \sum_{e', v' : (u, e', v') \in \mathcal{E}} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{v'}} \frac{\partial \mathbf{z}_{v'}}{\partial \mathbf{m}_{e'}} \frac{\partial \mathbf{m}_{e'}}{\partial \mathbf{x}_u} \\
&= \sum_{e', v' : (u, e', v') \in \mathcal{E}} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{v'}} \nabla_{\mathbf{m}_{e'}} \rho\left(\{\mathbf{m}_{e''} : (u'', e'', v') \in \mathcal{E}\}\right) \nabla_{\mathbf{x}_u} \phi_z\left(\mathbf{x}_u, \mathbf{y}_{v'}, \mathbf{w}_{e'}\right) \\
&= \sum_{e', v' : (v', e', u) \in \tilde{\mathcal{E}}} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{v'}} \nabla_{\mathbf{m}_{e'}} \rho\left(\left\{\mathbf{m}_{e''} : (v', e'', u'') \in \tilde{\mathcal{E}}\right\}\right) \nabla_{\mathbf{x}_u} \phi_z\left(\mathbf{x}_u, \mathbf{y}_{v'}, \mathbf{w}_{e'}\right)
\end{aligned}$$

We can see that for each node v, $\frac{\partial \mathcal{L}}{\partial \mathbf{y}_v}$ is only a function of its own features, the predecessors' features, and the features of inbound edges. Therefore, $\frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$ can be computed via a g-SpMM defined on graph $\mathcal{G}$.

Similarly, for each node u, $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_u}$ is only a function of its own features, the successors' features, and the features of outbound edges. It can be computed via a g-SpMM defined on the reverse graph $\tilde{\mathcal{G}}$ □

Proof of Theorem 1. The theorem can be proved trivially from the lemmas above, as the addition of two g-SDDMM functions on the same graph is still a g-SDDMM function on the same graph. The same holds for g-SpMM functions as well. □

APPENDIX C DATASET STATISTICS

Dataset	# Nodes	# Edges	# Node Features	# Edge Features
Node Classification				
REDDIT	232,965	11,606,919	602	0
OGBN-ARXIV	169,343	1,166,243	128	0
OGBN-PROTEIN	132,534	39,561,252	0	8
OGBN-PRODUCT	2,449,029	61,859,140	100	0
Link Prediction				
ML-100K	943 users 1,682 movies	100,000	943 (user) 1,682 (movie)	0
ML-1M	6,040 users 3,706 movies	1,000,209	6,040 (user) 3,706 (movie)	0
ML-10M	69,878 users 10,677 movies	10,000,054	69,878 (user) 10,677 (movie)	0
OGBL-CITATION	2,927,963	30,561,187	128	0
OGBL-PPA	576,289	30,326,273	58	0

Table 5: Statistics of all datasets used in Sec. 6

For ML-100K, ML-1M and ML-10M, we use separate one-hot encoding for user and movie nodes.

APPENDIX D EXPERIMENT CONFIGURATIONS

D.1 FULL GRAPH TRAINING

Here we list the hyper-parameter configurations used in comparing training speed between DGL and PyG (Sec. 6.1).

Node classification.

- The GraphSAGE model on REDDIT has two layers, each with 16 hidden size and the aggregator is summation.
- The GAT model on REDDIT has three layers, each with 16 hidden size and one attention head.
- The GraphSAGE model on OGBN-ARXIV has three layers, each with 256 hidden size and the aggregator is summation.
- The GAT model on OGBN-ARXIV has three layers, each with 16 hidden size and four attention heads.
- The R-GCN model on OGBN-PROTEIN has three layers with 32 hidden size. The graph has 8 edge features in the range of $[0, 1]$, which can be viewed as connectivity strength for 8 relations. The RGCN model takes the following formulation:
$$H^{(l+1)} = \sigma\left(\sum_{r=1}^{8} D_r^{-1} A_r H^{(l)} W_r^{(l)} + H^{(l)} W_0^{(l)}\right),$$
where $H^{(l)}$ is the node representations after the l-th RGCN layer, σ is the ReLU activation function, D_r is the degree matrix for relation r, A_r is the adjacency matrix for relation r, and $W_r^{(l)}, W_0^l$ are learnable weights. $H^{(0)}$ is the initial node features. Since the graph does not have raw node features, we use a scalar 1 for each node. The original A_r's are binary adjacency matrices and they become weighted in the case of OGBN-PROTEIN.

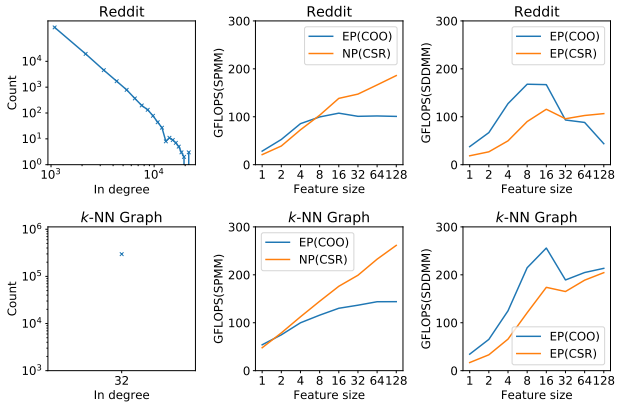


Figure 4: Throughput of SpMM and SDDMM with different parallel strategies, sparse formats on two input graphs. We fix the number of heads to 8 and vary the feature size of each head. The figures in the upper row are for the REDDIT dataset while the ones in the lower row are for the k-NN graph. NP stands for node parallel while EP stands for edge parallel.

Link prediction. The GCMC models on ML-100K, ML-1M and ML-10M all adopt an encoder-decoder architecture as proposed in the original paper. The input is a one-hot encoding of the movie and item nodes. The encoder has one graph convolution layer which projects the input encoding to a layer of 500 units with summation as the message aggregator, and one fully-connected layer which outputs a layer of 75 units. All the models use a bi-linear model as the decoder and the number of basis is set to two.

D.2 MINI-BATCH TRAINING

Node classification.

- For training with neighbor sampling on the REDDIT graph, we use a batch size of 1024 and the sampling fanouts are 25 and 10 from the first to the last layer. Both the GraphSAGE and the GAT models used have three layers and a hidden size of 16. The GAT model has 8 attention heads.
- For training with neighbor sampling on the OGBN-PRODUCT graph, we use a batch size of 1024 and sampling fanouts of 15, 10, 5 for GraphSAGE, and a batch size of 128 and sampling fanouts of 10, 10, 10 for GAT. Both the GraphSAGE and the GAT models have three layers with a hidden size of 256. The GAT model has 8 attention heads.
- For training with cluster sampling on the OGBN-PRODUCT graph, we first partition the graph into 15000 clusters. The training batch size is 32, meaning each mini-batch contains the induced subgraph of nodes from 32 clusters. Both the GraphSAGE and the GAT models have three layers with a hidden size of 256. The GAT model has 8 attention heads.

Link prediction. For all the experiments, we first partition the input graph into 15000 clusters. The training batch size is 256, meaning each mini-batch contains the induced subgraph of nodes from 256 clusters. All the models adopt an encoder-decoder architecture. The encoder models (i.e., GCN or GAT) all have three layers with a hidden size of 256. The GAT models all have one attention head. The decoder model predicts edges by a dot product between the representations of the incident nodes. By default, only one negative sample is generated per positive sample by corrupting one end-point of the edge.

APPENDIX E KERNEL SENSITIVITY TO GRAPH AND MODEL CONFIGURATION

We studied how graph structures and model configurations influence the speed of g-SpMM and g-SDDMM kernels and thus the choice of parallel strategies (i.e., node or edge parallel). We benchmarked a g-SpMM and a g-SDDMM kernel on two input graphs, the REDDIT graph from (Hamilton et al., 2017) and a nearest neighbor graph generated by (Qi et al., 2017), with varying feature sizes.

```python
from torch import nn

class SAGEConv(nn.Module):
  def __init__(self, in_feat, out_feat,
               feat_drop=0., activation=None):
    pass
  def forward(self, tensor):
    pass
```

(a) PyTorch

```python
from tensorflow.keras import layers

class SAGEConv(layers.Layer):
  def __init__(self, in_feats, out_feats,
               feat_drop=0., activation=None):
    pass
  def call(self, tensor):
    pass
```

(b) TensorFlow

Figure 5: Module classes inherit from different classes in PyTorch and TensorFlow.

```python
from torch import nn
fc = nn.Linear(in_feat, out_feat)
gain = nn.init.calculate_gain('relu')
nn.init.xavier_uniform_(fc, gain=gain)
```

(a) PyTorch

```python
import tensorflow as tf
from tensorflow.keras import layers
xinit = tf.keras.initializers.VarianceScaling(
            scale=np.sqrt(2), mode="fan_avg",
            distribution="untruncated_normal")
fc = layers.Dense(out_feats, kernel_initializer=xinit)
```

(b) TensorFlow

Figure 6: Create a fully connected sub-module in PyTorch and TensorFlow.

The two graphs have very different degree frequencies, with REDDIT (Figure 4 upper row) having power-law degree distribution while the k-NN graph (Figure 4 lower row) having a constant indegree equal to 32. The g-SpMM kernel is extracted from the Graph Attention Network (GAT) (Veličković et al., 2018) model; it multiplies a neighbor node's representation $\mathbf{x}_u$ with the attention weight α_e on the edge during message aggregation. With multiple attention heads (8 in our experiment), the formulation of each head h is as follows:

$$\mathbf{z}_{v,h} = \sum_{(u,e,v)\in\mathcal{E}} \alpha_{e,h}\mathbf{x}_{u,h}$$

The g-SDDMM kernel computes the attention weight by a dot-product of the source and destination nodes:

$$\alpha_{e,h} = \langle \mathbf{x}_{u,h}, \mathbf{x}_{v,h} \rangle, \forall (u,e,v) \in \mathcal{E}$$

The kernel throughput (in GFLOPS) measured on a NVIDIA V100 GPU is shown in Figure 4. It shows that the optimal choice of edge parallel or node parallel relies on graph structure, feature size and the sparse format in use. For SpMM, edge parallel is slightly better than node parallel for small feature size but eventually becomes worse when the feature size scales up due to the overhead from atomic aggregation. For SDDMM, edge parallel on COO outperforms CSR by a large margin up to feature size equal to 16 but again becomes worse afterwards because of the better memory locality of CSR.

APPENDIX F GUIDE FOR PORTING GNN MODELS ACROSS DEEP LEARNING FRAMEWORKS

DGL provides a framework-neutral design and allows users to develop GNN models on different deep learning frameworks, such as PyTorch, TensorFlow and MXNet. Assume a user finds an existing GNN model in one framework and wishes to port it to another one that s/he is familiar with. Such porting needs to address three categories of differences in the deep learning frameworks.

Porting models usually involves in three steps.

```python
from torch import nn


class SAGEConv(nn.Module):
  def __init__(self, in_feat, out_feat,
               feat_drop=0., activation=None):
    super(SAGEConv, self).__init__()
    src_feat, dst_feat = expand_as_pair(in_feat)
    self.feat_drop = nn.Dropout(feat_drop)
    self.activation = activation
    self.fc_self = nn.Linear(dst_feat, out_feat)
    self.fc_neigh = nn.Linear(src_feat, out_feat)
    gain = nn.init.calculate_gain('relu')
    nn.init.xavier_uniform_(
        self.fc_self.weight, gain=gain)
    nn.init.xavier_uniform_(
        self.fc_neigh.weight, gain=gain)

  def forward(self, graph, feat):
    feat_src = feat_dst = self.feat_drop(feat)
    graph.srcdata['h'] = feat_src
    graph.update_all(fn.copy_u('h', 'm'),
                     fn.mean('m', 'neigh'))
    h_neigh = graph.dstdata['neigh']
    rst = self.fc_self(feat_dst)
            + self.fc_neigh(h_neigh)
    return self.activation(rst)
```

(a) PyTorch

```tensorflow
import tensorflow as tf
from tensorflow.keras import layers

class SAGEConv(layers.Layer):
  def __init__(self, in_feats, out_feats,
               feat_drop=0., activation=None):
    super(SAGEConv, self).__init__()

    self.feat_drop = layers.Dropout(feat_drop)
    self.activation = activation
    xinit = tf.keras.initializers.VarianceScaling(
            scale=np.sqrt(2), mode="fan_avg",
            distribution="untruncated_normal")
    self.fc_self = layers.Dense(
            out_feats, kernel_initializer=xinit)
    self.fc_neigh = layers.Dense(
            out_feats, kernel_initializer=xinit)

  def call(self, graph, feat):
    feat_src = feat_dst = self.feat_drop(feat)
    graph.srcdata['h'] = feat_src
    graph.update_all(fn.copy_u('h', 'm'),
                     fn.mean('m', 'neigh'))
    h_neigh = graph.dstdata['neigh']
    rst = self.fc_self(feat_dst)
                + self.fc_neigh(h_neigh)
    return self.activation(rst)
```

(b) TensorFlow

Figure 7: The implementation of GraphSAGE in PyTorch and TensorFlow.

- Step 1 is to change model class inheritance. For example, when porting from Pytorch to TensorFlow, the model class should inherit from `tensorflow.keras.layer.Layer` instead of `torch.nn.Module`. Figure 5 shows such an example.
- Step 2 is to change the sub-modules used inside the model. These sub-modules are usually defined in the initialization method of the model class. Different frameworks usually define similar sub-modules but with different sub-module names and different arguments. They also initialize the parameters in the sub-modules differently. For example, the fully connected layer in Pytorch is defined in `nn.Linear` but it is defined in `layers.Dense` in TensorFlow. The arguments of the sub-modules are also different. We always need to define the input and output dimensions for the Pytorch sub-modules but only need to define the output dimensions for the TensorFlow sub-modules. In addition, PyTorch initializes parameters after the definition of `nn` modules, while TensorFlow specifies initialization method together with layer definition. Figure 6 shows an example of such differences.
- Step 3 is to replace the framework-specific operators. Similar to sub-modules, different frameworks define similar tensor operators but with different names and different input arguments. For example, matrix multiplication is defined in `tensorflow.matmul` in TensorFlow and is defined in `torch.matmul` in Pytorch.

Figure 7 shows a complete example of porting GraphSAGE from PyTorch to TensorFlow. We place the code side by side to contrast the key differences. As shown above, the PyTorch version of `SAGEConv` needs to inherit from `torch.nn.Module` while the TensorFlow version inherits from `tensorflow.keras.layers.Layers`. Most of the modifications are in the `__init__` function of the class, where we change the members defined as `nn` modules to TensorFlow's counterparts. In this example, the `forward` function (cf. `call` function in TensorFlow) only invokes DGL's message passing computation via `update_all`. Because no tensor operators are explicitly invoked, there are no modifications for tensor operators.